

40 — Security Hardening for Production APIs

What it is

Security hardening is the process of reducing the attack surface of a deployed application by closing known vulnerability patterns. For an API, this means ensuring that every request is authenticated, every input is validated, every database query is safe from injection, and the infrastructure itself limits what an attacker can do even if they find a foothold.

This document covers eight concrete hardening measures applied to the Bilingual Search API after a security audit flagged 2 CRITICAL and 3 HIGH severity issues:

Severity	Finding
CRITICAL	SQL injection via unvalidated <code>table</code> parameter
CRITICAL	Missing authentication on admin endpoints
HIGH	Hardcoded/default JWT secret
HIGH	No brute-force protection on login
HIGH	Missing role-based authorization (any logged-in user could access admin functions)

Each fix below maps to one or more of these findings.

Real-life analogy

Think of your API as an office building.

- **JWT hardening** is upgrading from a master key that's taped under the doormat (a default secret everyone knows) to a unique key that's cut fresh each day. If someone copies it, it only works until the next shift change (token expiry).

- **Brute-force protection** is the security guard who says "you've tried the wrong badge 5 times -- come back in 5 minutes" instead of letting someone stand at the door trying combinations forever.
- **SQL injection prevention** is like a mailroom that only accepts packages addressed to known departments ("category_vectors" or "company_vectors"). If someone writes "category_vectors; DROP TABLE users" on the label, the mailroom rejects the package before it ever reaches the building.
- **Input validation** is a size limit on those packages -- no one needs to send a 10,000-word query, so we cap it at a reasonable length to prevent abuse.
- **Security headers** are the signs on the building: "No photography", "Authorized personnel only", "No drones". Browsers read these headers and enforce restrictions on behalf of the user.
- **Rate limiting** is a turnstile at the entrance -- it lets a steady flow of people through but blocks anyone trying to rush the door 100 times per minute.
- **Connection pooling** is a shared fleet of company cars instead of buying a new car for every employee's single trip and scrapping it after. Reuse saves resources and prevents exhaustion.
- **Docker hardening** is the principle of least privilege -- instead of giving every employee an admin badge, you give them the minimum access they need to do their job. Running as `appuser` instead of `root` means that even if someone breaks in, they can't access system-level controls.

Why we used it here

The Bilingual Search API serves as the backend for both the public search interface and the Admin UI. A security audit of the codebase (before these changes) found:

1. **SQL injection (CRITICAL):** The `table` parameter in the chunking endpoint was a raw string that flowed directly into an f-string SQL query. An attacker could pass `"users; DROP TABLE category_vectors--"` and the database would execute it. This is the single most dangerous class of web vulnerability.
2. **Missing auth / weak auth (CRITICAL + HIGH):** Some admin endpoints had no authentication. The JWT secret was a hardcoded default string (`"change-me-to-a-random-string"`), meaning anyone who read the source code could forge valid tokens. And even with a valid token, there was no role check -- any authenticated user could perform admin operations.

3. **No brute-force protection (HIGH)**: The login endpoint had no rate limiting. An attacker could try millions of username/password combinations without being slowed down.

These are not theoretical risks. SQL injection and authentication bypass are consistently in the OWASP Top 10 (the industry's authoritative list of web application security risks). The fixes below are standard, well-understood patterns that every production API should implement.

Code walkthrough

1. JWT Authentication Hardening

File: `search_api/routers/auth.py`

The JWT secret is the cryptographic key used to sign and verify tokens. If an attacker knows this key, they can forge tokens for any user, including admins.

```
# Read the secret from the environment variable.
# If it's empty or still the placeholder default, we don't trust it.
JWT_SECRET = os.getenv("JWT_SECRET", "")

if not JWT_SECRET or JWT_SECRET == "change-me-to-a-random-string":
    import secrets
    # Generate a 32-byte cryptographically random string, base64-encoded.
    # This is 256 bits of entropy – infeasible to brute-force.
    JWT_SECRET = secrets.token_urlsafe(32)
    logger.warning(
        "JWT_SECRET not set or is default – generated ephemeral secret"
    )
    # "Ephemeral" means tokens signed with this secret will be invalid
    # after the next server restart. This is intentional: it forces
    # operators to set a real secret in production, rather than silently
    # running with a known key.
```

Why ephemeral? In development, convenience matters -- you don't want the app to crash just because you forgot to set an env var. But in production, tokens should survive restarts (so users don't get logged out on every deploy). The warning log makes it obvious that the secret needs to be configured properly.

Token expiry was reduced from 24 hours to 8 hours:

```
TOKEN_EXPIRY_HOURS = 8 # was 24 – shorter window limits damage from a stolen token
```

If an attacker steals a token (from a log file, a browser's local storage, or a network capture), they have at most 8 hours to use it instead of 24.

Role-based authorization now checks the `role` claim inside the JWT payload:

```
def require_admin(
    credentials: HTTPAuthorizationCredentials = Depends(security),
) -> str:
    """Dependency that verifies the caller has admin role."""
    payload = jwt.decode(
        credentials.credentials, # the raw JWT string from the Authorization header
        JWT_SECRET,
        algorithms=[JWT_ALGORITHM],
    )
    # Previously, any valid token was enough. Now we check the role claim.
    if payload.get("role") != "admin":
        raise HTTPException(
            status_code=403, # 403 Forbidden, not 401 – the user IS authenticated,
            detail="Admin role required", # they just don't have permission.
        )
    return payload["sub"] # return the username for downstream use
```

Brute-force protection tracks failed login attempts per IP+username combination:

```
LOGIN_MAX_ATTEMPTS = 5 # max attempts allowed in the window
LOGIN_LOCKOUT_SECONDS = 300 # 5-minute sliding window

# In-memory dict: key is "ip:username", value is list of attempt timestamps
_login_attempts: dict[str, list[float]] = defaultdict(list)

def _check_login_rate(key: str) -> None:
    now = time.time()
    cutoff = now - LOGIN_LOCKOUT_SECONDS

    # Keep only attempts within the last 5 minutes
    attempts = [t for t in _login_attempts[key] if t > cutoff]
    _login_attempts[key] = attempts

    # If 5 or more attempts in the window, reject immediately
    if len(attempts) >= LOGIN_MAX_ATTEMPTS:
        raise HTTPException(
            status_code=429, # 429 Too Many Requests – standard HTTP code for rate
            detail=f"Too many login attempts. Try again in "
```

```
)                f"{LOGIN_LOCKOUT_SECONDS // 60} minutes.",
```

The key is `{client_ip}:{username}`. This means an attacker can't just switch usernames to bypass the limit from the same IP, and a legitimate user isn't locked out because someone else on a different IP is attacking their account.

2. SQL Injection Prevention

Files: `search_api/routers/chunking.py`, `search_api/models/requests.py`

This was the most critical fix. Here's what was happening before:

```
# BEFORE – VULNERABLE:
table: str = Query("category_vectors")

# Later in the endpoint handler:
cursor.execute(f"SELECT ... FROM {table} WHERE ...")
# An attacker could send: table = "users; DROP TABLE category_vectors--"
# The database would see: SELECT ... FROM users; DROP TABLE category_vectors-- WHERE ...
# That semicolon starts a new statement. The -- comments out the rest. Disaster.
```

The fix is deceptively simple but extremely effective:

```
# AFTER – SAFE:
table: Literal["category_vectors", "company_vectors"] = Query("category_vectors")
```

`Literal` is a Python typing construct that Pydantic (FastAPI's validation layer) enforces at request time. If a client sends `table=anything_else`, Pydantic returns a 422 Validation Error *before the endpoint code even runs*. The database never sees the malicious input.

This is called **allowlisting** (as opposed to blocklisting). Instead of trying to detect every possible attack pattern (which is impossible), we define exactly what values are allowed and reject everything else.

3. Input Validation

File: `search_api/models/requests.py`

Every search query field now has a maximum length:

```
class CategorySearchRequest(BaseModel):
    query: str = Field(
```

```

...,          # required (no default)
min_length=1,  # reject empty strings
max_length=1000, # reject oversized payloads
description="Free-text search query",
)

```

Why does this matter? Two reasons:

1. **Database protection:** A 100,000-character query would force PostgreSQL to process an enormous `tsvector` comparison and a huge embedding computation. This could tie up the database for seconds, enabling a denial-of-service attack.
2. **LLM cost protection:** The RAG endpoint sends the query to an LLM. Longer queries cost more tokens. A malicious user could rack up API costs by sending massive queries. The `max_length=2000` on `RAGRequest.query` puts a ceiling on this.

4. Security Response Headers

File: `search_api/main.py`

These headers are added to every HTTP response via the existing request metadata middleware:

```

# Prevents the browser from guessing the content type.
# Without this, a browser might interpret a JSON response as HTML
# and execute any <script> tags inside it.
response.headers["X-Content-Type-Options"] = "nosniff"

# Prevents your page from being embedded in an iframe on another site.
# This blocks "clickjacking" – where an attacker overlays your page
# with invisible elements to trick users into clicking things.
response.headers["X-Frame-Options"] = "DENY"

# Activates the browser's built-in XSS (cross-site scripting) filter.
# Modern browsers have moved to Content-Security-Policy, but this
# still helps with older browsers.
response.headers["X-XSS-Protection"] = "1; mode=block"

# Controls how much URL information is sent when navigating away.
# "strict-origin-when-cross-origin" means: send the full URL to same-origin
# requests, but only the origin (no path/query) to other domains.
response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"

# Blocks the page from accessing device hardware it doesn't need.

```

```
# An API has no business accessing the camera, microphone, or GPS.
response.headers["Permissions-Policy"] = (
    "camera=(), microphone=(), geolocation=()"
)

# Tells browsers and proxies not to cache the response.
# Since our API returns search results that may include sensitive data,
# we don't want cached copies sitting on shared machines or CDN nodes.
response.headers["Cache-Control"] = "no-store"
```

Each header is a defense-in-depth measure. No single header prevents all attacks, but together they significantly reduce the attack surface.

5. Rate Limiting with Memory Leak Fix

File: `search_api/main.py`

Global rate limiting caps how many requests any single IP can make per minute:

```
# Configurable via environment variable. Default: 60 requests per minute.
RATE_LIMIT_RPM = int(os.getenv("RATE_LIMIT_RPM", "60"))

# In-memory tracking: IP -> list of request timestamps
_request_log: dict[str, list[float]] = defaultdict(list)

@app.middleware("http")
async def rate_limit(request, call_next):
    client_ip = request.client.host
    now = time.time()
    window_start = now - 60 # 60-second sliding window

    # --- Memory leak prevention ---
    # Without this, _request_log grows forever as new IPs appear.
    # After 10,000 unique IPs, we clean up any that haven't made
    # a request in the last 60 seconds.
    if len(_request_log) > 10_000:
        stale = [
            ip for ip, ts in _request_log.items()
            if not ts or ts[-1] < window_start
        ]
        for ip in stale:
            del _request_log[ip]

    # Trim this IP's history to only the current window
    timestamps = _request_log[client_ip]
    _request_log[client_ip] = [t for t in timestamps if t > window_start]
```

```

# Check if they've exceeded the limit
if len(_request_log[client_ip]) >= RATE_LIMIT_RPM:
    return JSONResponse(
        status_code=429,
        content={"detail": "Rate limit exceeded. Try again later."},
    )

# Record this request
_request_log[client_ip].append(now)
return await call_next(request)

```

The memory leak fix is important. In a production environment, your API might receive requests from millions of unique IPs (especially if it's under a bot attack). Without cleanup, the `_request_log` dictionary would grow unboundedly until the process runs out of memory and crashes. The 10,000-entry threshold triggers a cleanup pass that removes IPs that haven't been seen recently.

6. Connection Pooling

File: `search_api/db.py`

Before this change, every API request opened a new TCP connection to PostgreSQL and closed it when done. A TCP connection involves a 3-way handshake, TLS negotiation, and PostgreSQL authentication -- all before a single query runs. Under load, this means hundreds of connections being created and destroyed per second.

```

_pool: pool.ThreadedConnectionPool | None = None

def _get_pool():
    """Lazily initialize the connection pool (singleton)."""
    global _pool
    if _pool is None or _pool.closed:
        _pool = pool.ThreadedConnectionPool(
            minconn=2,      # keep at least 2 connections ready at all times
            maxconn=int(os.getenv("DB_POOL_MAX", "10")), # never exceed 10
            dsn=DATABASE_URL,
        )
    return _pool

```

The clever part is the wrapper that makes pooling transparent to all existing code:

```

class _PooledConnection:
    """Wrapper that returns connections to the pool instead of closing them."""

    def __init__(self, conn, p):

```

```

        self._conn = conn      # the real psycopg2 connection
        self._pool = p        # reference to the pool

    def close(self):
        # Instead of conn.close() which destroys the TCP connection,
        # we return it to the pool for reuse.
        self._pool.putconn(self._conn)

    def cursor(self, *args, **kwargs):
        return self._conn.cursor(*args, **kwargs)

    def commit(self):
        return self._conn.commit()

    def __getattr__(self, name):
        # Any method not explicitly defined above is forwarded to
        # the real connection. This means rollback(), autocommit, etc.
        # all work transparently.
        return getattr(self._conn, name)

    def get_connection():
        """Get a connection from the pool (drop-in replacement for psycopg2.connect()).
        p = _get_pool()
        return _PooledConnection(p.getconn(), p)

```

The `__getattr__` trick is what makes this a zero-change upgrade for the 28 caller sites. Every place in the codebase that calls `conn = get_connection()` and later `conn.close()` now transparently uses the pool. The callers don't know or care that the connection is being recycled instead of destroyed.

7. Docker Hardening

Files: `search_api/Dockerfile`, `bilingual_etl/Dockerfile`

By default, Docker containers run as `root`. This means that if an attacker exploits a vulnerability in your application, they have root access inside the container. While container isolation limits the blast radius, running as a non-root user is a critical defense-in-depth measure.

```

# Create a user with no password, no home directory, and no login shell.
# This is the minimum footprint for a service account.
RUN adduser --disabled-password --no-create-home appuser

# Switch to this user for all subsequent commands (and the container entrypoint).
USER appuser

```

The `.dockerignore` files prevent sensitive files from being included in the Docker image:

```
.env          # contains real API keys – must never be baked into an image
.env.*        # any environment variants
tests/        # test code has no place in production images
docs/         # documentation adds image size for no runtime benefit
__pycache__/  # compiled Python bytecode – will be regenerated
*.pyc
```

Without `.dockerignore`, a `COPY . .` instruction would include everything in the build context, including `.env` files with real API keys. Anyone who pulls the Docker image could extract those secrets.

8. Secret Management

The `.gitignore` ensures that `.env` files (which contain real API keys for Groq, Gemini, Jina, etc.) are never committed to version control:

```
.env
```

The `.env.example` file provides the template with safe defaults:

```
JWT_SECRET=change-me-to-a-random-string
GROQ_API_KEY=
GEMINI_API_KEY=
```

The combination of these three practices forms a defense chain:

1. `.gitignore` prevents accidental commits of secrets
2. `.env.example` documents what env vars are needed without exposing real values
3. The ephemeral JWT secret (from section 1) ensures the app never silently runs with a known key

How to explain this in an interview / to a teammate

"We ran a security audit on the API and found two critical issues: a SQL injection vulnerability where a raw string flowed into an f-string query, and missing auth on admin endpoints. We fixed the injection by switching the parameter type to a

`Literal` so Pydantic rejects any value not in our allowlist before it ever reaches the database. For auth, we hardened the JWT flow -- auto-generating ephemeral secrets when no real one is configured, adding role-based checks, and rate-limiting login attempts to block brute-force attacks. On top of that, we added security response headers, global rate limiting with a memory-leak-safe cleanup, connection pooling to stop creating a new TCP connection per request, and locked down Docker to run as a non-root user. Every fix maps to a standard OWASP mitigation pattern."